



Introduction to Computer Science

Unit 8: Booleans

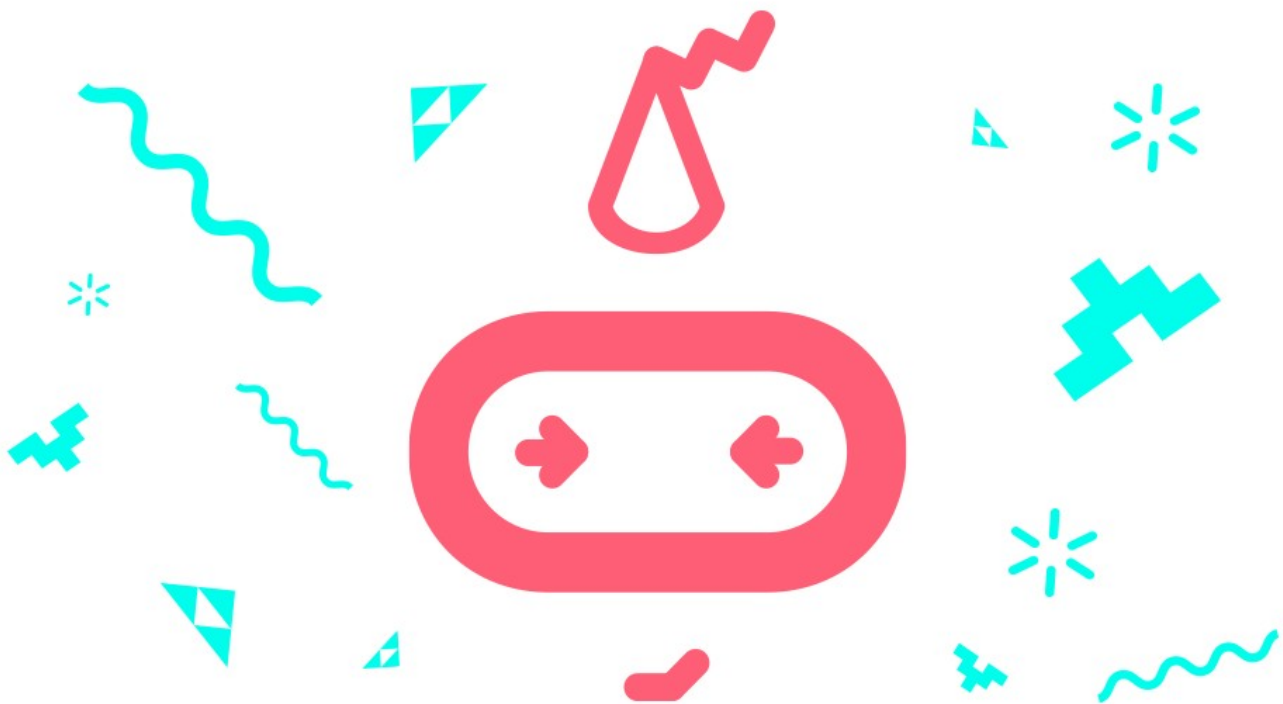
Student workbook
makecode.microbit.org



Table of Contents

Overview.....	3
---------------	---

Unit summary.....	3
Learning goals.....	3
Lesson A: Understanding Booleans.....	4
Lesson B: Code a double coin flipper.....	7
Lesson C: Project Boolean.....	16
Glossary of key terms.....	26



Overview

Unit summary

This unit introduces the use of the Boolean data type to control the flow of a program, keep track of the status of the program, and to include or exclude certain conditions. In an unplugged activity, you will write pseudocode to simulate two coins being tossed at the same time. In the coding activity, you will take the pseudocode from the unplugged activity and use it to code your micro:bit. In the project, you will code your own unique program using Booleans and other blocks you've explored and learned in the previous units.

Learning goals

During this unit, you will:

- Understand what Booleans and Boolean operators are and why and when to use them in a program.
- Learn how to create a Boolean, set the Boolean to an initial value, and change the value of the Boolean within a micro:bit program.
- Learn how to use the random true or false block.
- Apply the above knowledge and skills to create a unique program that uses Booleans and Boolean operators as an integral part of the program.

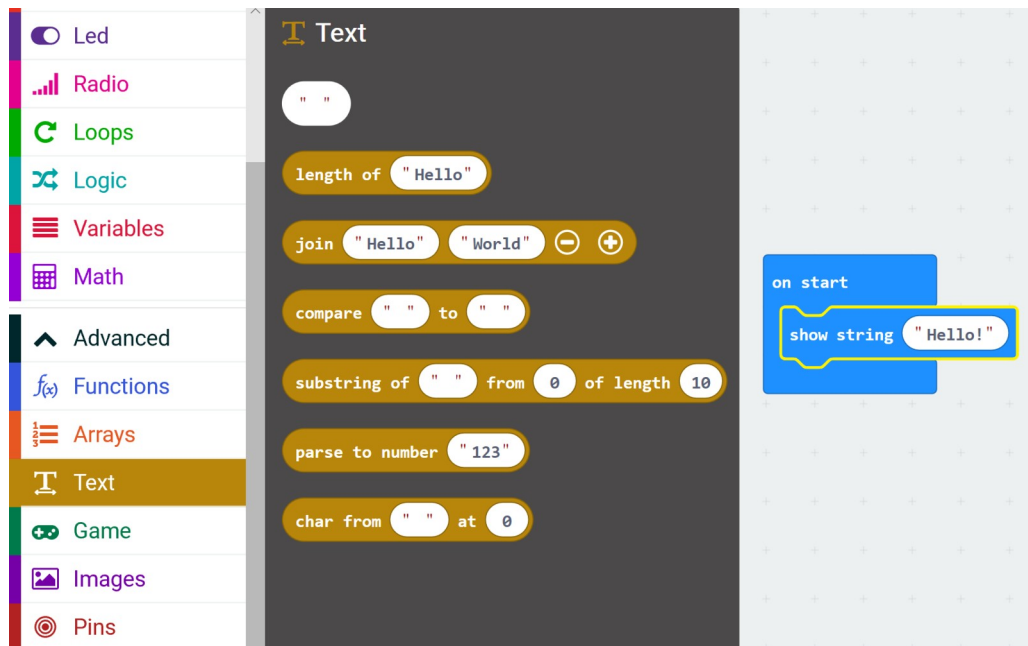
Use this space to write your questions and ideas

Lesson A: Understanding Booleans

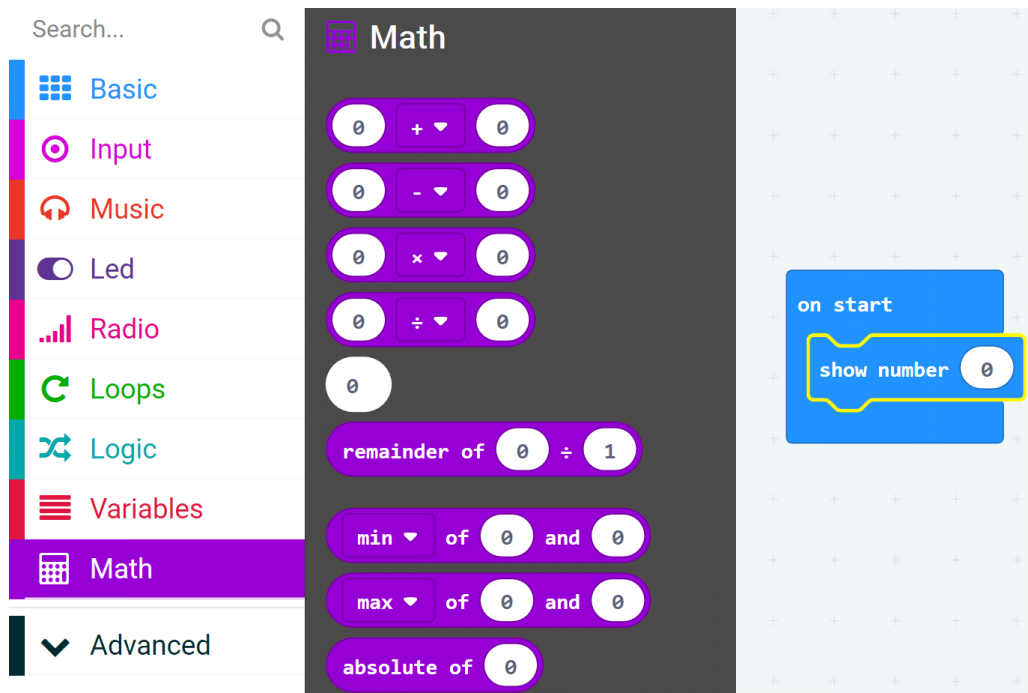
Overview: Booleans

There are several different data types used in computer programming. You have already used two of these types:

- String types for text and alphanumeric characters



- Number types for integer and decimal values



George Boole



George Boole (November 2, 1815 – December 8, 1864) was an English mathematician, educator, philosopher, and logician. He worked in the fields of differential equations and algebraic logic and is best known as the author of *The Laws of Thought* (1854) which contains Boolean algebra.

Boolean is another type of data. A *Boolean* data type has only two values: *true* or *false*. In true binary fashion, these two values can be represented by the numbers: 1 = true, and 0 = false.

Booleans are useful in programming for decision making, often deciding when certain functions and parts of programs should start or stop running. They're also used in database searches.

Boolean operators: AND, OR, and NOT

To make working with Booleans useful for solving more complex decisions and searches, you can connect two or more Booleans into one decision statement. To do this, you can use what are known as Boolean operators. The three most common and the ones you will use with the micro:bit are: And, Or, and Not.

These operators can be used in conditionals and loops, like:

- If condition A is true AND condition B is true
- If condition A is true OR condition B is true
- While event A has NOT happened

AND

(Condition A AND Condition B): For this expression to evaluate as true, both conditions in the expression need to be true. So, if both Condition A AND Condition B are true, the expression will evaluate as (or return) true.

OR

(Condition A OR Condition B): For this expression to evaluate as true, only one of the conditions in the expression needs to be true. If Condition A is true, the expression will return true regardless of whether Condition B is true or false. If Condition B is true, the expression will return true regardless of whether Condition A is true or false.

NOT

NOT can be used when checking that a condition is false (or not true). For example:

- (NOT Condition A and Condition B) evaluates as true only if Condition A is false and Condition B is true.
- (Condition A and NOT Condition B) evaluates as true only if Condition A is true and Condition B is false.
- (NOT Condition A and NOT Condition B) evaluates as true only if both Condition A and Condition B are true.

NOT is also useful when using a loop. For example, you can use a NOT to check:

- While button A is NOT pressed, continue to run this code...

Note: “False” can be thought of as equivalent to “NOT true”.

Lesson B: Code a double coin flipper

Coding activity: Double coin flipper

Create a program using Boolean variables and operators, and the pseudocode from Lesson A, to code a double coin flipper program.

The pseudocode:

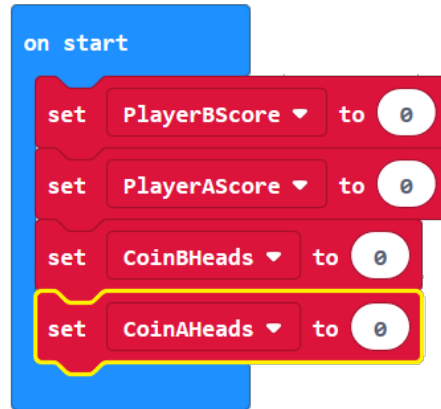
1. Use the random function to get a true/false value for Coin A.
2. Use the random function to get a true/false value for Coin B.
3. Compare the current values of Coin A and Coin B.
 - If the current true/false values of Coin A and Coin B are the same, add a point to Player A's score.
 - Otherwise, the current true/false values of Coin A and Coin B must be different, so add a point to Player B's score.
4. When you are done with your double coin flipping, show the final scores for each player.

Initialize the variables

In Microsoft MakeCode, start a new project and name it something like: Double coin flipper. You can leave the 'on start' block in the coding Workspace but can delete the 'forever' loop block.

1. For the first step, create your variables. From the Variables Toolbox drawer, use the Make a variable button to create each of the following:
 - CoinAHeads
 - CoinBHeads
 - PlayerAScore
 - PlayerBScore

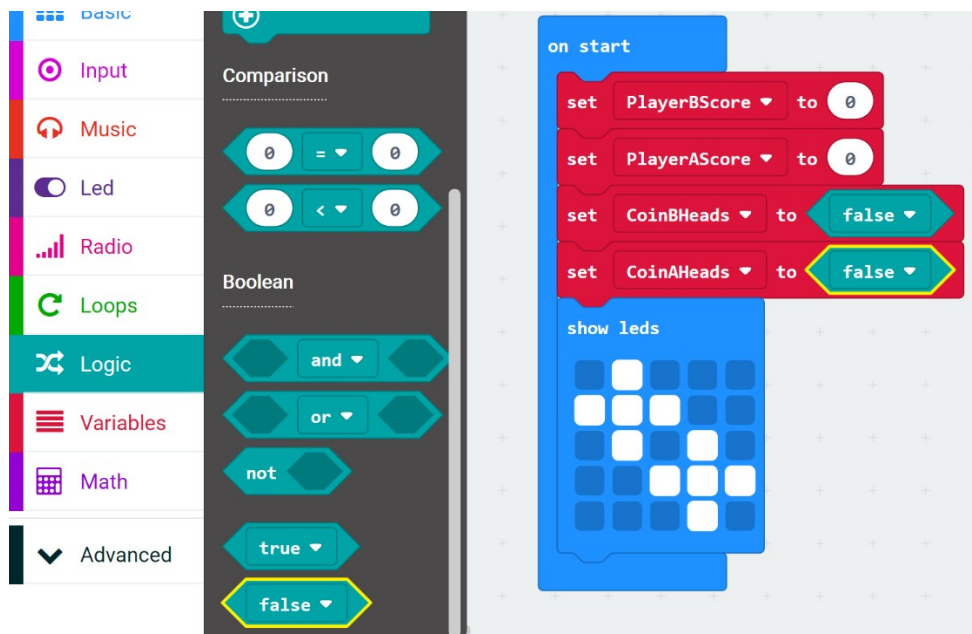
- Now you need to initialize the variable values. Put four 'set (variable)' blocks inside the 'on start' block and use the dropdown menu to set the variable for each block to each of the new variables.



The initial value of a variable is the value the variable will hold each time the program starts. By default:

- a string variable is initialized to an empty string: ""
 - a number variable is initialized to: 0
 - a Boolean is initialized to: false
- Leave the number variables at 0 and initialize the Boolean variables to 'false'. You can find the 'false' hexagon blocks under the Boolean section of the Logic Toolbox drawer.

Then, add an image for the start screen, so the user knows the program has started and is ready. In the example below, the image is intended to look like two coins.

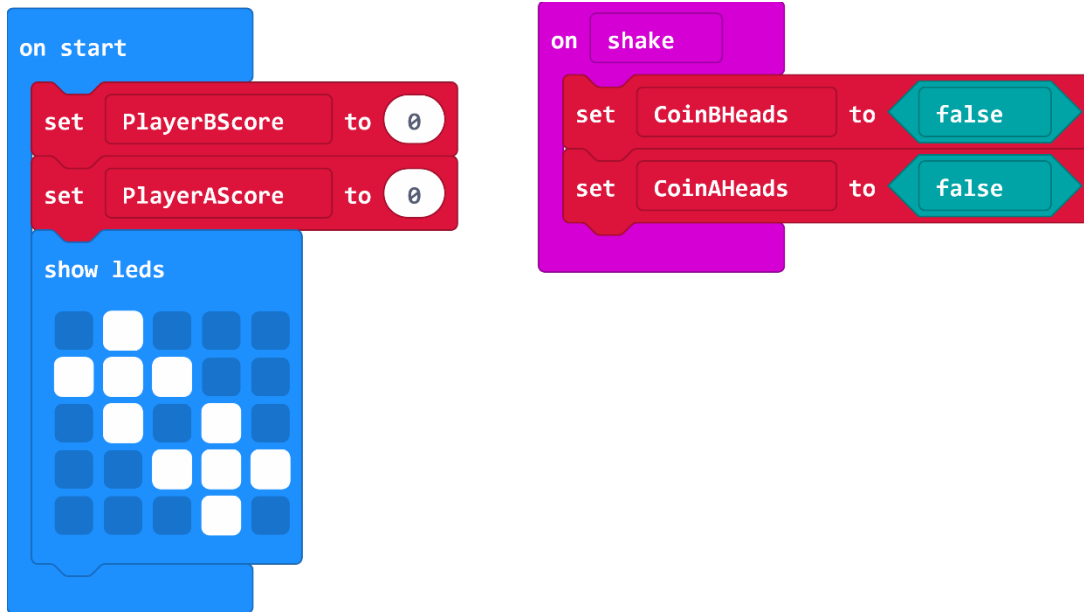


Random coin flips

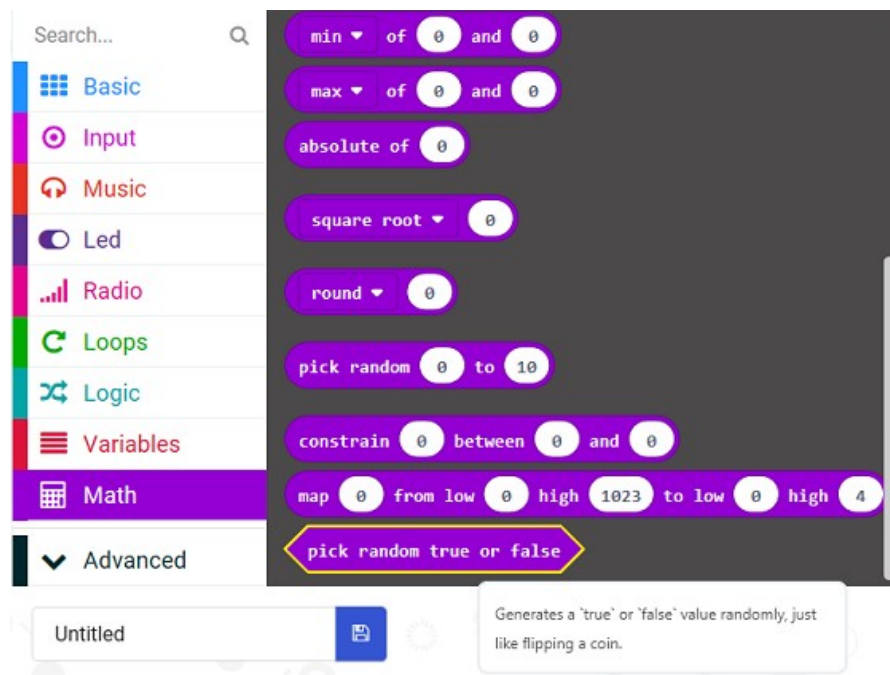
Use the micro:bit's accelerometer to mimic tossing a coin. The *accelerometer* measures the acceleration of your micro:bit; this component senses when the micro:bit is moved. It can also detect other actions like shake,

tilt, and free fall. When the player shakes the micro:bit, you will code the micro:bit to give each of your Boolean variables a random true/false value.

4. From the Input Toolbox drawer, drag an 'on shake' block to the coding Workspace. Drag the two 'Set CoinAHeads' and 'Set CoinBHeads' blocks from the 'on start' block into the 'on shake' block.



5. From the Math Toolbox drawer, drag two 'pick random true or false' blocks to the coding Workspace. Hover over this 'pick random' block and note that its pop-up description mentions coin flipping!



6. Drop one each of these 'pick random' blocks to the 'set' variable blocks in the 'on shake' block.



That completes the first two steps of your pseudocode:

- a. Use the random function to get a true/false value for Coin A.
- b. Use the random function to get a true/false value for Coin B.

Compare current values of both coins

Now that the virtual CoinA and CoinB have been virtually flipped, you need to compare the outcomes to see if they are the same or different, which is the next step of your pseudocode:

Compare the current values of Coin A and Coin B.

- a. If the current true/false values of Coin A and Coin B are the same, add a point to Player A's score.
 - b. Otherwise, if the current true/false values of Coin A and Coin B are different, add a point to Player B's score.
7. From the Logic Toolbox drawer, drag an 'if...then...else' block to the coding Workspace. Then, drag the 'if...then...else' block into the 'on shake' block under the 'set' variable blocks.



Because you were able to visualize your blocks as you wrote your pseudocode, you already know what blocks you will use. You also know that you have simplified your code as much as possible! You can simply add this to your current code and provide user feedback by adding some visuals.

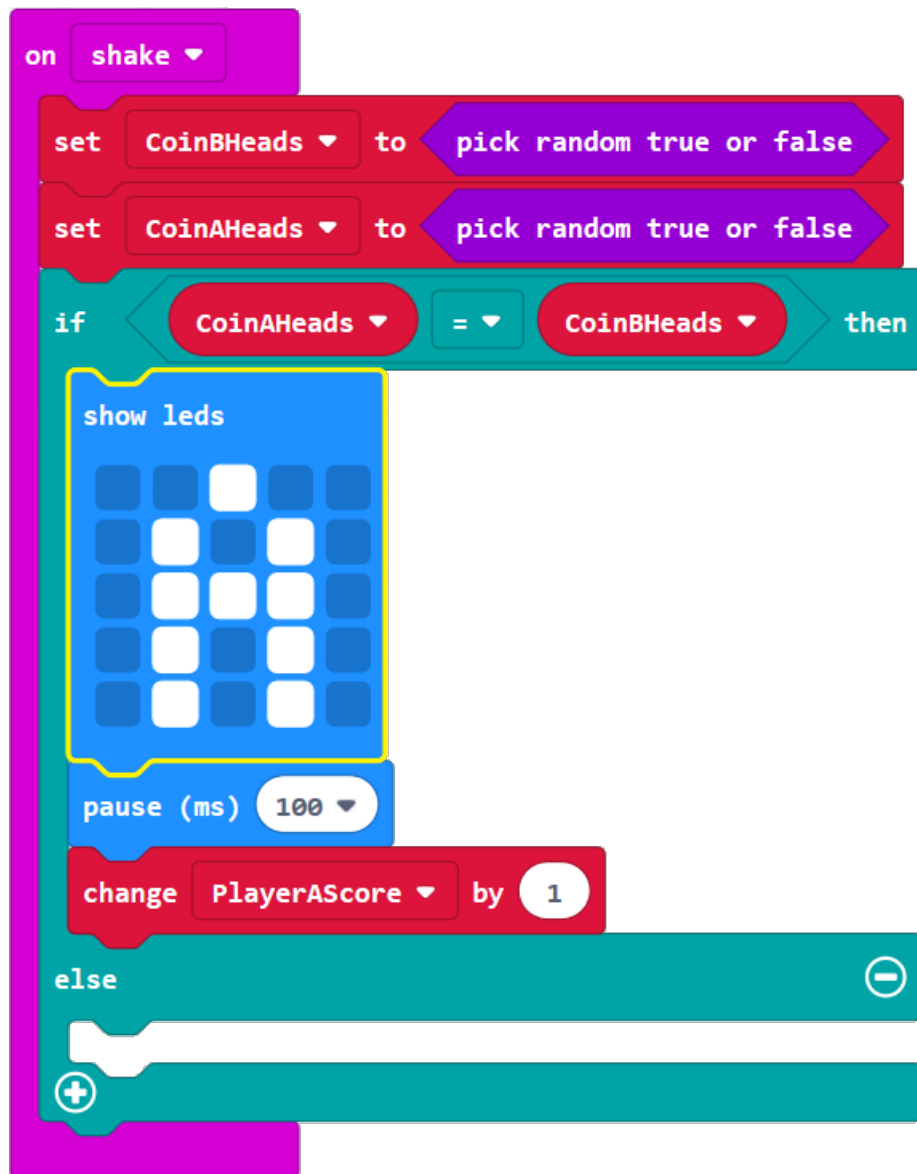
- From the Logic Toolbox drawer, in the Comparison section, drag a '0 = 0' hexagon block to the coding Workspace and replace the 'true' hexagon in the 'if...then...else' block.

Then, from the Variables Toolbox drawer, drag a 'CoinAHeads' variable to replace the first 0. You could drag the 'CoinBHeads' variable to replace the second 0, but another option is to duplicate the 'CoinAHeads' variable, snap it into the second 0, and use the dropdown menu to select 'CoinBHeads' option.

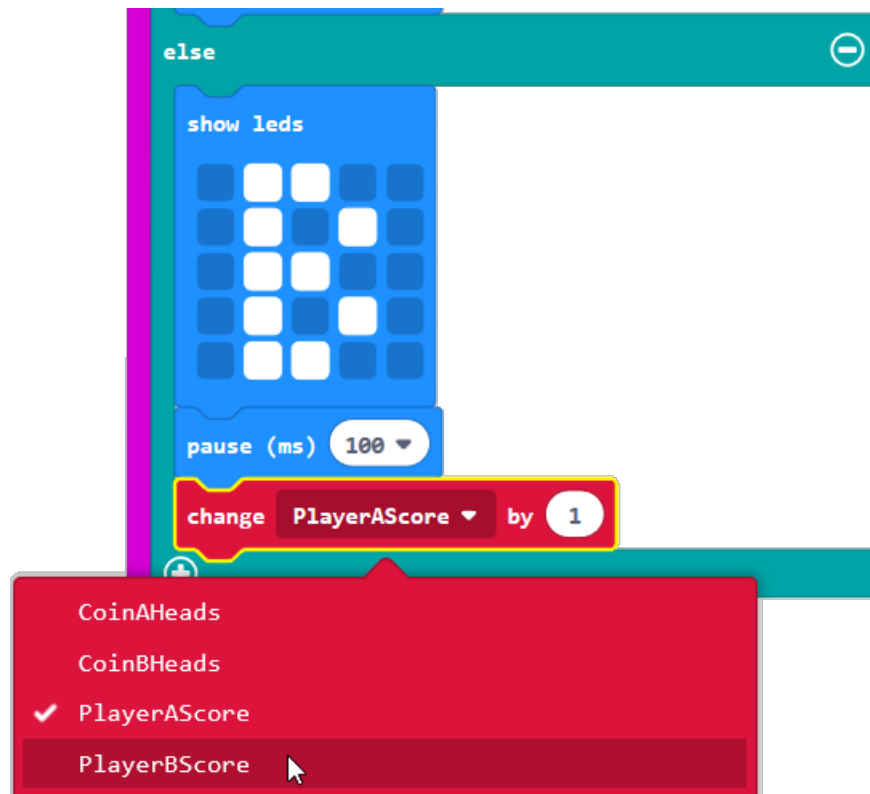


9. To add a point to Player A's score, go to the Variables Toolbox drawer and drag a 'change (variable) by 1' block into the 'then' option. Make sure it's set to the 'PlayerAScore' variable.

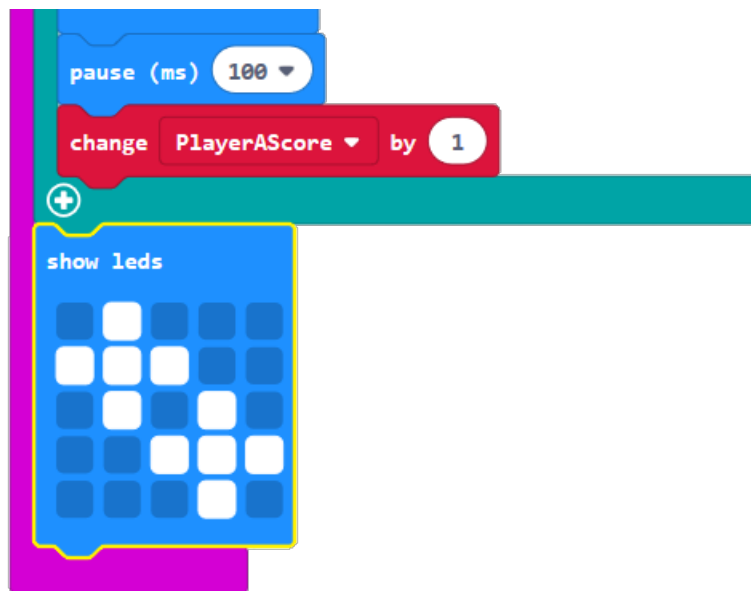
Then, to give a visual cue that Player A got the point, go to the Basic Toolbox drawer, drag a 'show leds' block and 'pause' block onto the coding Workspace and connect them into the 'then' option above the 'change PlayerBScore by 1' block. In the 'show leds' block, select the boxes to show the letter A.



10. Do the same for Player B in the 'else' option. Instead of dragging the needed blocks from the Toolbox drawers, duplicate the blocks and change the 'show leds' block and 'change PlayerAScore' variable accordingly.



11. To signify the end of the on-shake coin toss, show the two-coin image on the micro:bit again. From the Basic Toolbox drawer, drag a 'show leds' block to the coding Workspace and connect it below the conditional blocks. Select the boxes to show the two coins.



12. Now, test the code in the Simulator to make sure it works as intended.

Show each player's score

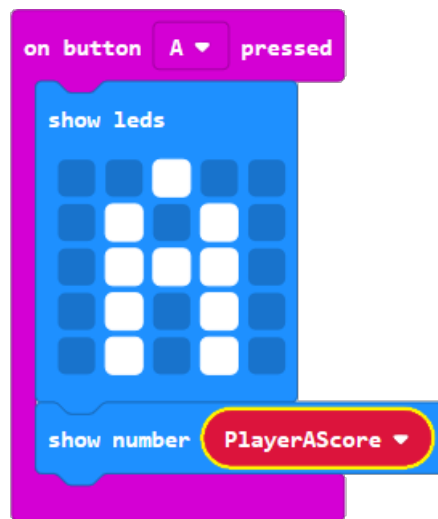
To finish your program, you'll complete the last step of the pseudocode.

When players are done with their double coin flipping, show the final scores for each player.

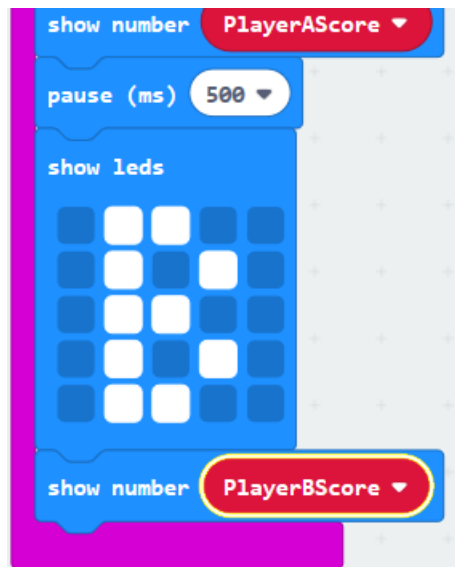
You'll use button A to do this.

13. From the Input Toolbox drawer, drag an 'on button A pressed' block to the coding Workspace. Duplicate the 'show leds' block with the letter A and connect it inside the 'on button A pressed' block. Then, from the Basic Toolbox drawer, drag the 'show number' block to the Workspace and connect it below the 'show leds' block.

14. From the Variables Toolbox drawer, drag a 'PlayerAScore' variable to replace the 0.



15. Let's add a pause before showing Player B's score. Use the dropdown menu to select 500 ms. Then, follow the previous steps to show Player B's score. Try to complete this without going into the Toolbox! Hint: Use duplicates and dropdown menus.



The final steps are to add another 500 ms pause and have the micro:bit show the double coins to signify it's ready for another on-shake coin toss.

Test, download and play!

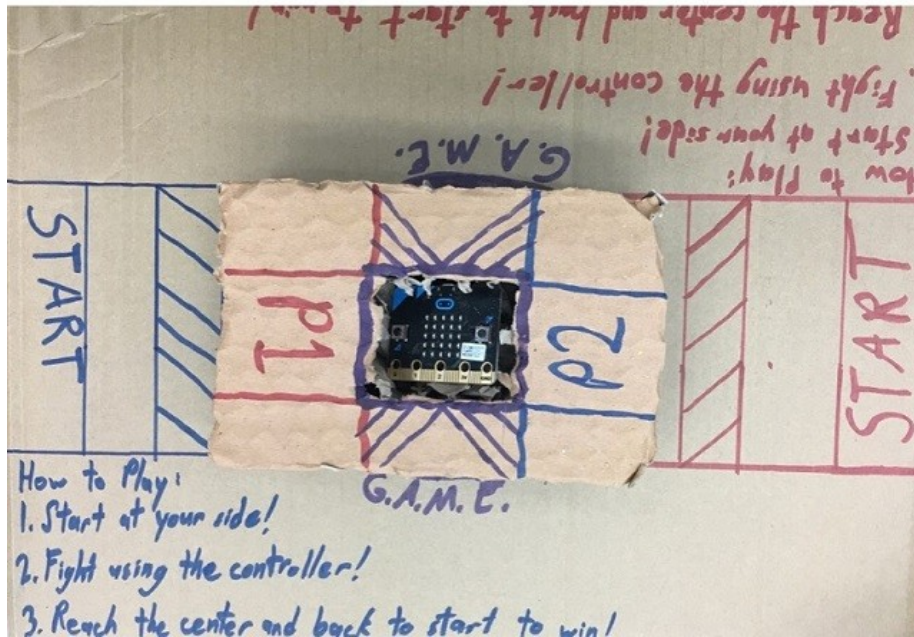
16. Test your code on the Simulator. Then, download to the micro:bits and try it out! Play a few more rounds of the Double Coin Flip using your new micro:bit Double Coin Flipper!

Use this space to write your questions and ideas

Lesson C: Project Boolean

Activity: Project Boolean

In this project, you will come up with a micro:bit program that uses Boolean variables, Boolean operators, and possibly the random function.



Project expectations

Follow the design thinking approach and make sure your project meets these specifications:

- More than two Boolean variables are implemented in a meaningful way
- The micro:bit program uses Booleans in a way that is integral to the program
- The program compiles and runs as intended and includes meaningful comments in code
- Provide the written Reflection Diary entry

The design thinking process:

1. **Empathize** by learning more about your target audience.
2. **Define**: understand and identify your audience's problems or needs.
3. **Ideate**: Brainstorm several possible creative solutions.
4. **Prototype**: Construct rough drafts or sketches of your ideas.
5. **Test** your prototype solutions and refine until you come up with the final version.

Use this space to start your design thinking process:

Input

Take a moment to think of all the different inputs available through the micro:bit.

Available inputs:

- Acceleration
- Light level
- Rotation
- Button is pressed
- Compass heading
- Temperature
- Running time
- On shake
- On button pressed
- On logo down
- On logo up
- On pin pressed
- On screen down
- On screen up
- Pin is pressed

Project ideas

Use Boolean variables and/or random values to create:

- A board game, game pieces, and a holder for the micro:bit (or improve your board game from Unit 3: Variables)
- A mod of some sort to a current/existing board game
- A micro:bit version of a Magic Eight Ball
- A micro:bit version of the “Red light, green light” unplugged activity from Unit 4: Conditionals

Project examples

1. Sunscreen monitor

The micro:bit is attached to a bottle of sunscreen and provides information about the temperature and if you need sunscreen:

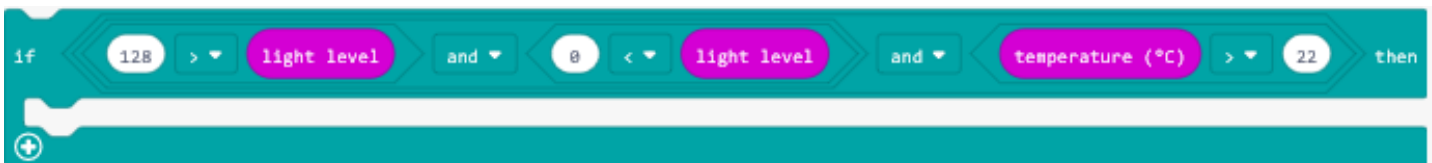
- When you shake the micro:bit, it reports the current temperature in degrees Fahrenheit.
- Button A displays an animation to tell you whether or not you should use sunscreen (on sunny or cloudy days but not at night or indoors.)
- Button B measures the light level and if it is above 70 degrees AND very bright, it will display a sun icon. If it is above 70 degrees and less bright, it will display a cloudy symbol. If it is dark, it will display a nighttime icon.



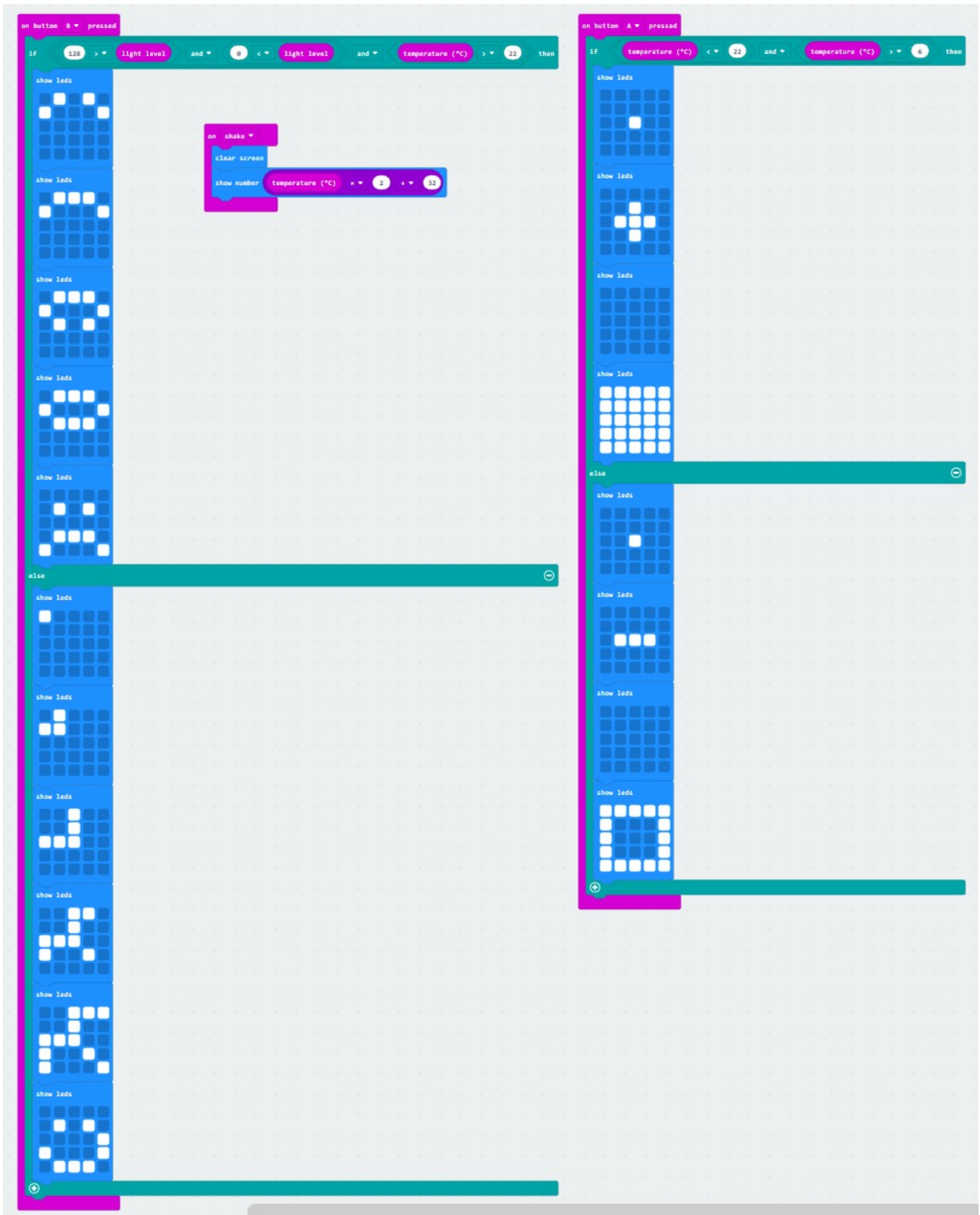
Check it out in action at youtu.be/VmD-dcZZQFc (0:18)

Coding detail

Following is a portion of the code that uses Boolean operations because both light level AND temperature must be high in order to trigger the sun icon:



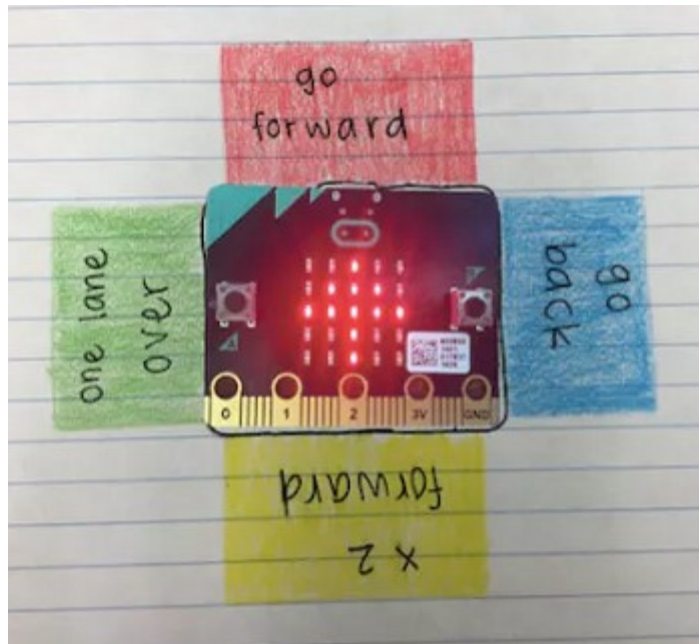
Complete code



Solution link: makecode.microbit.org/_Atd9Wti3MiUj

2. Board game with arrows

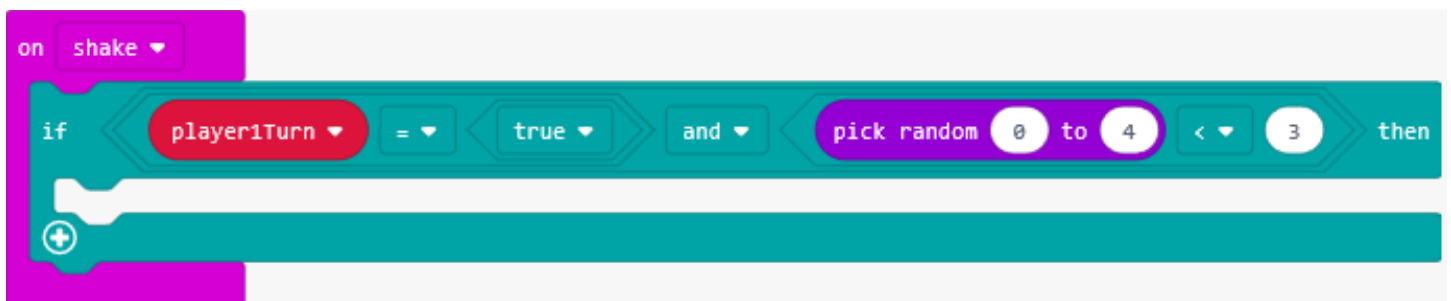
This is an example of a board game in which the micro:bit displays an arrow pointing in a random direction. The paper legend indicates different actions the player must take, and it uses a Boolean variable to keep track of whose turn it is.



Coding detail

Following is a portion of the board game's code that uses a Boolean variable to determine whose turn it is.

- If player1Turn is false, then it's player 2's turn.
- A random number is generated to show the arrow seventy-five percent of the time (for values of 0, 1, or 2).



Complete code

Part 1 – code from beginning to 'clear screen' block

```
on shake
  if playerTurn == true and pick random 0 to 4 < 3 then
    clear screen
    set delay to 0
    while delay < 500
      do
        show leds
        change delay by 50
        pause (ms) delay
    end while
    change delay by 50
    pause (ms) delay
    show leds
    change delay by 50
    pause (ms) delay
    show leds
    change delay by 50
    pause (ms) delay
    show leds
    change delay by 50
    pause (ms) delay
    set playerTurn to false
  else if playerTurn then
    show string "Crash!"
    set playerTurn to false
  else if pick random 0 to 4 < 3 then
    clear screen
```

on start

```
set delay to 0
set spin to 0
set playerTurn to true
```

Part 2 – code from 'clear screen' block to end

```
clear screen
set delay to 0
while delay < 500
  do
    show leds
    change delay by 50
    pause (ms) delay
    show leds
    change delay by 50
    pause (ms) delay
    show leds
    change delay by 50
    pause (ms) delay
    show leds
    change delay by 50
    pause (ms) delay
    set playerTurn to true
  end do
else
  show string "Crash!"
  set playerTurn to true
end while
```

Solution link: makecode.microbit.org/_1mY4wq4KPYiq

Use this space to start your design thinking process:

Project scoring rubric

Assessment elements	1	2	3	4
Boolean	No Boolean variables are implemented.	At least one Boolean variable is implemented in a meaningful way.	At least two Boolean variables are implemented in a meaningful way.	More than two Boolean variables are implemented in a meaningful way.
micro:bit program	micro:bit program lacks three or more of the required elements.	micro:bit program lacks two of the required elements.	micro:bit program lacks one of the required elements.	micro:bit program: ■ Uses Booleans in a way that is integral to the program ■ Compiles and runs as intended ■ Uses meaningful comments in code

Reflection Diary

Expectations

Write a reflection of about 150–300 words addressing the following points:

- How did you incorporate Boolean variables into your micro:bit program?
- How did you incorporate Boolean operators into your micro:bit program?
- Describe something in your project that you are proud of.
- What feedback did your beta testers give you? How did that help you improve your design?
- If you had more time to work on this project, describe what you might add or change.
- Publish your MakeCode program and include the link.

Diary entry scoring rubric

Assessment elements	1	2	3	4
Diary entry	Diary entry is missing three or more of the required elements.	Diary entry is missing two of the required elements.	Diary entry is missing one of the required elements.	Diary entry addresses all elements, including: ▪ Brainstorming ideas ▪ Construction ▪ Programming ▪ Beta testing

Glossary of key terms

Boolean: A data type that can be either true or false. A Boolean condition is a condition that evaluates to either true or false.